# Lab 5 Data Structure and File i/o

Lab Report

Author: Gagan Adhikari

Kristianstad University

December 24, 2025

# Contents

**1 Introduction** **3**

**2 Task 1: Memory-Efficient Data Structure** **4**
2.1 Structure Design . . . . . 4
2.2 Memory Calculation . . . . . 5
2.3 Structure Diagram . . . . . 5

**3 Task 2: Simulation Application** **6**
3.1 Simulation Logic . . . . . 6
3.2 Main Simulation Code . . . . . 6
3.3 Example Output . . . . . 7

**4 Task 3: Finding Maximum Temperature and Humidity** **8**
4.1 Algorithm . . . . . 8
4.2 Code for Maximum Search . . . . . 8
4.3 Result Display . . . . . 9

**5 Flowchart and pseudo code** **10**
5.1 Flowchart . . . . . 10
5.2 pseudo code . . . . . 11

**6 Conclusion** **12**

# Authorship Statement

I, **Gagan Adhikari**, declare that this lab report is the result of my own original work. All external sources used have been properly cited and acknowledged. This report has not been submitted previously for any academic or professional purpose.

AI assistance was used to make latex code look better and add some extra packages like to show c code looks better.

# Chapter 1

# Introduction

This report presents the design and implementation of a memory-efficient sensor logging system for a resource-limited embedded environment. The system simulates five sensors operating over a period of 1000 seconds, generating a new sensor record every 3 seconds. For a certain fun, I have added sleep function to delay display process for 3 sec. Each record contains:

- Sensor ID
- Timestamp
- Temperature
- Humidity
- Battery status

The primary goals are:

1. Design a memory-efficient data structure using bit-fields.
2. Implement a simulation that logs sensor data to a binary file.
3. Identify the sensor with the highest temperature and humidity.

# Chapter 2

# Task 1: Memory-Efficient Data Structure

## 2.1 Structure Design

To minimize memory usage, bit-fields and `pragma pack` were used to eliminate padding. And use of macros: The final structure is shown below:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <time.h>
#include <windows.h>

#define temp_rand   ((rand() % 256) - 128)
#define humid_rand  (rand() % 101)
#define batt        (rand() % 2)
#define c_1         1000
#define clock_step  3
#define sen_id      5
#define sleep       3000
#define in8       8
#define in7       7
#define in12      12
#define in1       1
```

Figure 2.1: macros and package

```
#pragma pack(push,1)
struct SensorRecord
{
    uint8_t  sensorID    : 8;
    uint16_t timestamp   : 12;
    int8_t   temperature: 8;
    uint8_t  humidity    : 7;
    uint8_t  battery     : 1;
}; // total = 36 bits = 5 bytes
#pragma pack(pop)
```

## 2.2 Memory Calculation

- Sensor ID: 8 bits
- Timestamp: 12 bits
- Temperature: 8 bits
- Humidity: 7 bits
- Battery: 1 bit

$$\text{Total bits} = 8 + 12 + 8 + 7 + 1 = 36 \text{ bits}$$

$$\text{Total bytes} = \frac{36}{8} = 4.5 \approx 5 \text{ bytes}$$

The `sizeof()` operator confirms the structure occupies exactly 5 bytes.

```
Size of SensorRecord = 5 bytes
```

Figure 2.2: output of size of struct

## 2.3 Structure Diagram

```c
#pragma pack(push,1)
struct SensorRecord
{
    uint8_t  sensorID    : in8;   // 1 byte
    uint16_t timestamp   : in12;  // 1.5 bytes
    int8_t   temperature: in8;   // 1 byte
    uint8_t  humidity    : in7;   // <1 byte
    uint8_t  battery     : in1;   // <1 byte
}; // total = 36 bits = 5 bytes
#pragma pack(pop)

int read_size()
{
    struct SensorRecord rec;
    printf("Size of SensorRecord = %zu bytes\n", sizeof(rec));
    return 0;
}
```

Figure 2.3: sensor-struct

# Chapter 3

# Task 2: Simulation Application

## 3.1 Simulation Logic

The simulation runs from 0 to 1000 seconds in steps of 3 seconds. At each step, all 5 sensors generate a new record. The data is written to a binary file and printed on screen.

## 3.2 Main Simulation Code

```
for(uint16_t Clock = 0; Clock < 1000; Clock += 3)
{
    for(uint8_t id = 0; id < 5; id++)
    {
        rec.sensorID    = id;
        rec.timestamp   = Clock;
        rec.temperature = temp_rand;
        rec.humidity    = humid_rand;
        rec.battery     = batt;

        record_file(fp,&rec);
        print_record(&rec);
    }
    printf("\n");
    Sleep(3000);
}
```

## 3.3 Example Output

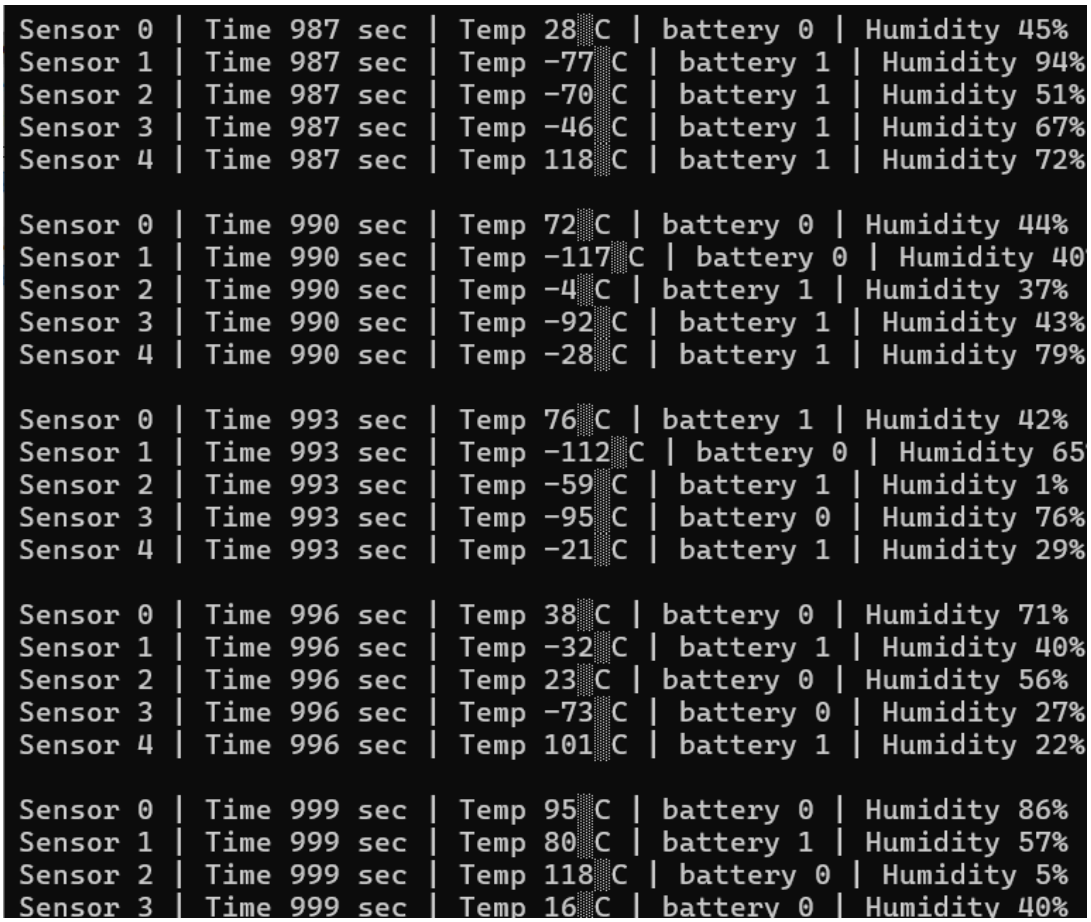



Figure 3.1: Example console output

# Chapter 4

# Task 3: Finding Maximum Temperature and Humidity

## 4.1 Algorithm

The binary file is scanned sequentially. Two records are tracked:

- `maxtemp` – record with highest temperature
- `maxhum` – record with highest humidity

## 4.2 Code for Maximum Search

```
while(fread(&rec, sizeof(rec),1,fp))
{
    if(rec.temperature > maxtemp.temperature)
        maxtemp = rec;
    if(rec.humidity > maxhum.humidity)
        maxhum = rec;
}
```

## 4.3 Result Display

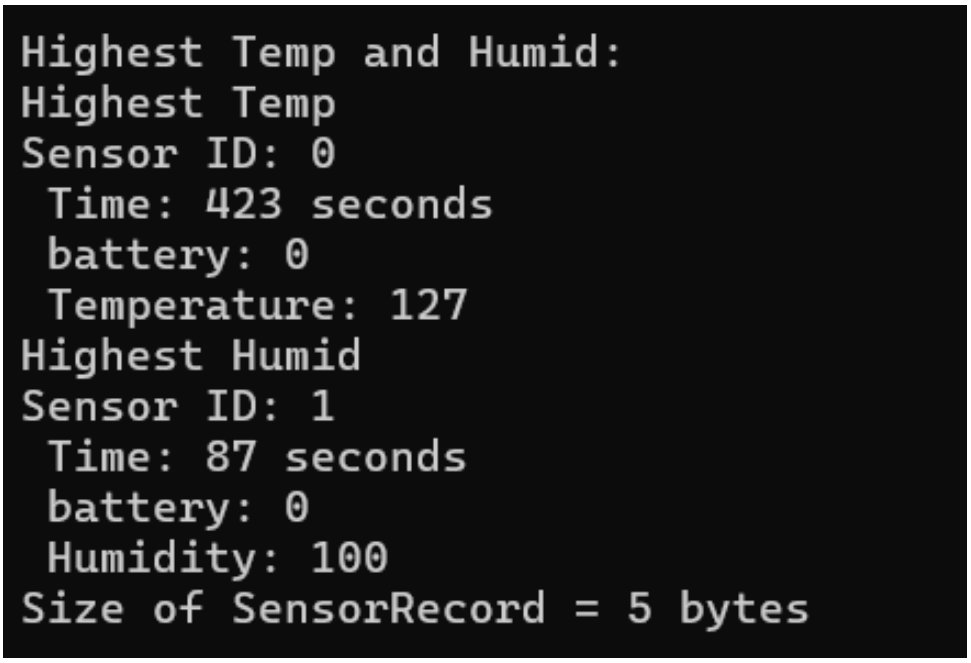



Figure 4.1: Screenshot of highest temperature and humidity results

# Chapter 5

# Flowchart and pseudo code

## 5.1 Flowchart

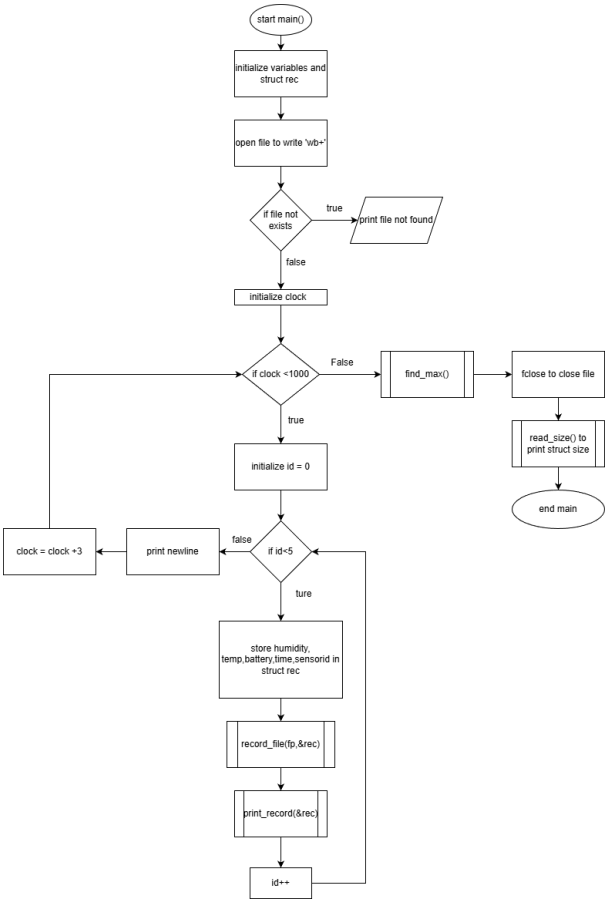



Figure 5.1: flowchart for main

## 5.2 pseudo code

**Algorithm 1** Main Function

```
1: procedure MAIN
2:     Initialize random seed
3:     Open file sensor.bin in write/update mode
4:     for Clock = 0 to c_1 step clock_step do
5:         for id = 0 to sen_id − 1 do
6:             rec.sensorID ← id
7:             rec.timestamp ← Clock
8:             rec.temperature ← random value in [−128, 127]
9:             rec.humidity ← random value in [0, 100]
10:            rec.battery ← random value in {0, 1}
11:            Write record to file
12:            Print record to console
13:        end for
14:        Print newline
15:    end for
16:    Find and print highest temperature and humidity records
17:    Close file
18: end procedure
```

# Chapter 6

# Conclusion

This project demonstrates how memory-efficient data structures and modular C programming techniques can be applied in resource-limited embedded systems. The use of bit-fields and packed structures reduced the memory footprint to only 5 bytes per sensor record. The simulation successfully generated and written sensor data, and the analysis functions correctly identified the maximum temperature and humidity readings.